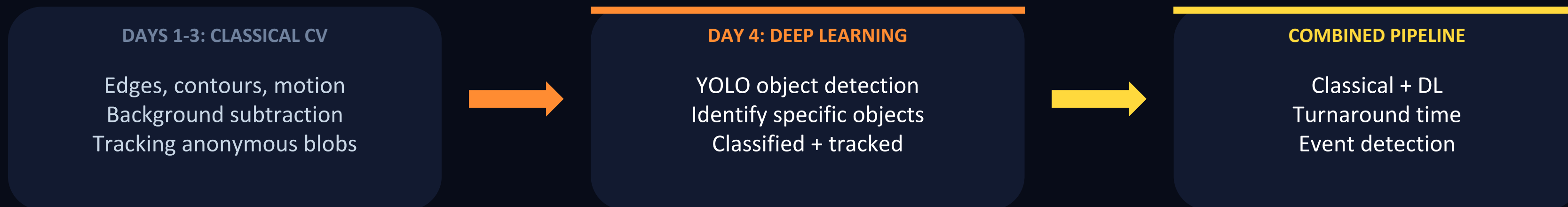


DAY 4 | COMPUTER VISION WEEK

Deep Learning for Computer Vision

& The Turnaround Pipeline

Adding intelligence — from motion blobs to identified, classified objects



Why Deep Learning for Computer Vision?

Classical CV detects motion and shapes — but it can't identify WHAT things are

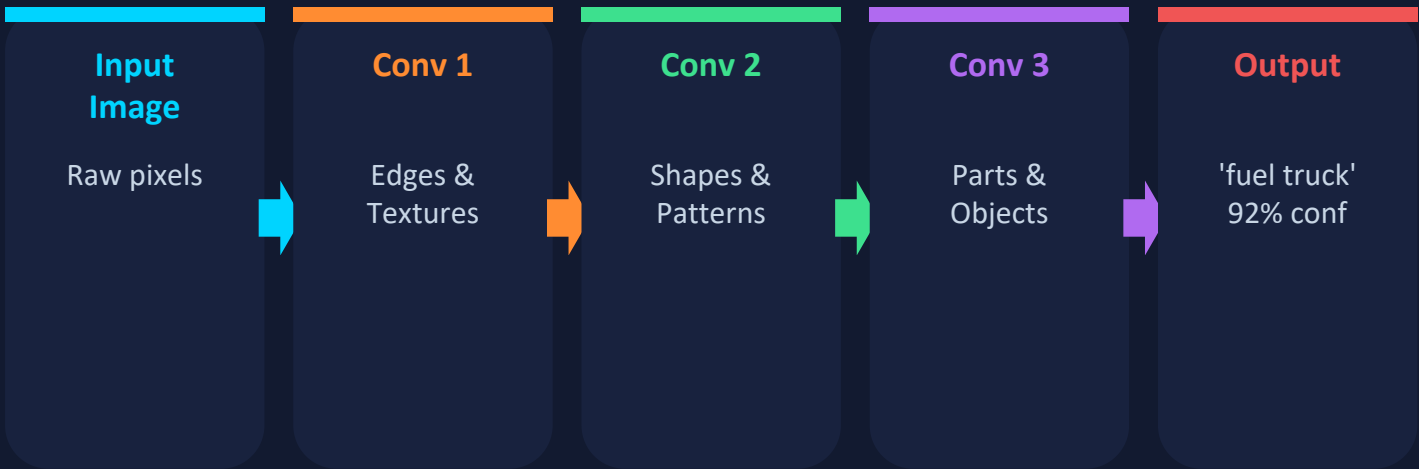
The Limitation of Classical CV

Over Days 1-3 we built a powerful toolkit. But it has a fundamental gap — it can't understand what objects are:

- Background subtraction detects a 'moving blob'
→ But is it a fuel truck or a baggage cart?
- Contour analysis finds a 'large rectangle'
→ But is it the airplane or the terminal?
- Optical flow shows 'something moved left'
→ But was it a vehicle departing or arriving?

How CNNs Learn to See

Convolutional Neural Networks learn visual features automatically from thousands of labeled examples:



Classical CV answers WHERE — Deep Learning answers WHAT.

Classical CV + Deep Learning = Best of Both Worlds

Task	Best Approach	Day	Why
Image preprocessing	Classical CV	1	Grayscale, blur, resize — rules-based
Edge / contour analysis	Classical CV	2	Canny, findContours — deterministic
Background subtraction	Classical CV	3	MOG2 — statistical model of background
Object classification	Deep Learning	4	CNN identifies WHAT an object is
Object detection	Deep Learning	4	YOLO finds what + where (bounding box)
Object tracking	Hybrid	3+4	YOLO + DeepSORT — identified tracking

1b

Three Levels of Visual Understanding

Each level adds more detail about what's in the image



Classification

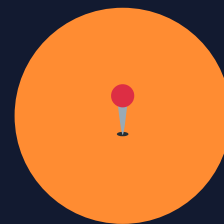
What is in this image?

One label for the entire image.
No location information.

Input: image of a tarmac
Output: 'fuel truck' (92%)

Good for: sorting images
by category.

```
model = YOLO('yolo11m-cls.pt')
```



Object Detection

What is here AND where?

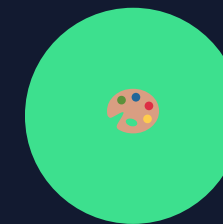
Labels + bounding boxes.
Know what each object is
and where it's located.

Input: airport frame
Output: 'fuel truck' at
(x=100, y=200, w=150, h=80)

Best for: our project!

```
model = YOLO('yolo11m.pt')
```

← Our project uses this level



Segmentation

Pixel-perfect boundaries?

Every pixel labeled.
Exact shape outline for
each detected object.

Input: airport frame
Output: mask showing every
pixel of the fuel truck

Best for: precise shape
analysis.

```
model = YOLO('yolo11m-seg.pt')
```

Object Detection — Output & Metrics

What a detector produces and how we measure its quality

What a Detector Returns

For each detected object, you get three pieces of data:

```
detection = {  
  'class': 'fuel_truck',  
  'confidence': 0.92,  
  'bbox': [x1, y1, x2, y2]  
}
```

class = what is it? confidence = how sure? bbox = where?

Two Approaches to Detection

Two-Stage: Faster R-CNN

1) Propose regions 2) Classify each region.
Higher accuracy, but slower (5-10 FPS).

One-Stage: YOLO / SSD

Single pass → all detections at once. Real-time! (30-180 FPS)

Detection Metrics — How We Measure Quality

Precision

Of all detections made,
how many were correct?

$$TP / (TP + FP)$$

Ideal: 100%

High = few false alarms

Recall

Of all real objects,
how many were found?

$$TP / (TP + FN)$$

Ideal: 100%

High = missed nothing

mAP

Mean Average Precision
across all classes.

Overall quality metric.

Ideal: 100%

Standard benchmark

IoU

Intersection over Union.
How well does predicted
box overlap the real box?

Ideal: 1.0

≥ 0.5 = correct detection

FPS

Frames per second.
How fast can the model
process video?

Ideal: Higher

Real-time ≥ 30 FPS

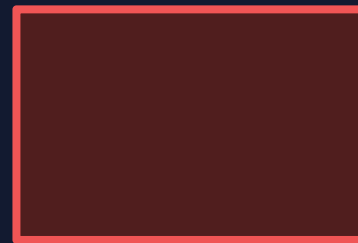
2b

IoU — Intersection over Union (Visual Guide)

The key metric for judging if a detection's bounding box is correct

How IoU Works

IoU measures overlap between the predicted bounding box and the ground truth (real) box.



IoU = 0.0

No Overlap



IoU = 0.5

50% Overlap



IoU = 1.0

Perfect Match

Formula

$$\text{IoU} = \text{Area of Overlap} / \text{Area of Union}$$

Where Union = Area(Predicted) + Area(Ground Truth) - Overlap

Ranges from 0.0 (no overlap) to 1.0 (perfect)

Detection = Correct When:

1. $\text{IoU} \geq 0.5$ (box overlaps enough with ground truth)
AND
2. Class label matches ('truck' predicted, 'truck' is real)

If either condition fails → false positive (wrong detection).

Non-Maximum Suppression (NMS)

When multiple boxes detect the same object, NMS keeps only the best one. It compares all overlapping boxes ($\text{IoU} > \text{threshold}$), keeps the highest confidence, and removes the rest. This prevents duplicate detections of the same object. • Parameter: iou threshold (default 0.7)

YOLO — You Only Look Once

One forward pass through the network → all detections. That's why it's fast.

How YOLO Works — 4 Steps

1

DIVIDE

Split image into an $S \times S$ grid (e.g. 13×13). Each cell is responsible for detecting objects whose center falls in it.

2

PREDICT BOXES

Each grid cell predicts B bounding boxes + confidence scores. Confidence = how likely this box contains any object.

3

PREDICT CLASSES

Each cell also predicts class probabilities: $P(\text{airplane})$, $P(\text{truck})$, etc. Combined with box confidence for final score.

4

NMS CLEANUP

Non-Maximum Suppression removes duplicate detections. Keep highest confidence box, delete overlapping ones.

Why Single Pass = Fast

Traditional detectors (Faster R-CNN) do TWO passes:

Pass 1: Propose ~2,000 regions of interest

Pass 2: Classify each region separately

→ 2,000+ neural network evaluations per image!

YOLO does ONE pass:

The entire image goes through the network once.

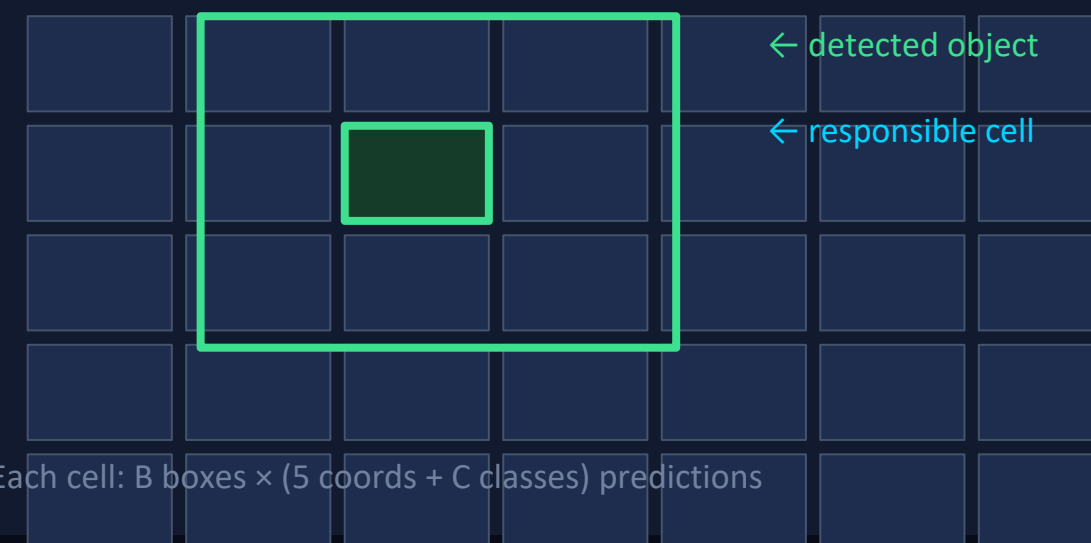
All grid cells predict simultaneously in parallel.

→ 1 neural network evaluation per image.

This is 50-100× faster than two-stage detectors.

Grid Prediction — Visual

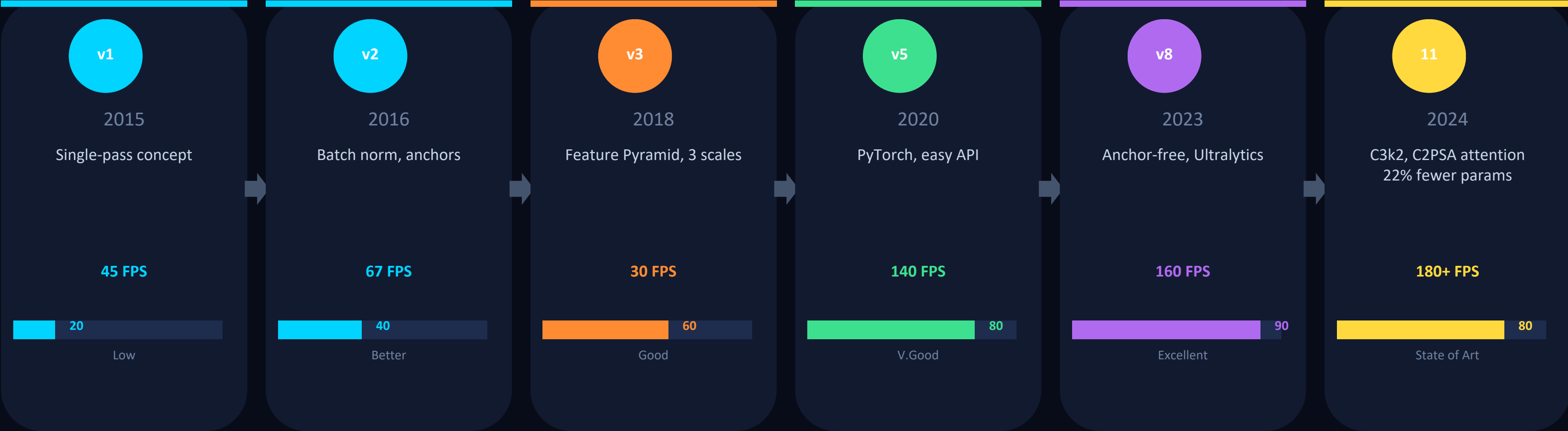
Image divided into cells, each predicting boxes:





YOLO Evolution — 2015 to YOLO11

Each version faster, more accurate, easier to use



We use YOLO11 — the latest, fastest, and most accurate. Same simple Ultralytics API.

Pre-Trained on COCO — 80 Classes

Vehicles

car, bus, truck, airplane, motorcycle, bicycle, boat, train

People

person (single class — but very robust detection)

Airport-relevant

airplane ✓ truck ✓ car ✓ person ✓ bus ✓

Not in COCO

jet bridge ✗ fuel truck ✗ baggage cart ✗ pushback ✗



YOLO11 Model Sizes — Speed vs Accuracy Tradeoff

Five sizes from edge devices to maximum accuracy

Nano (11n) Params: 2.6M mAP: 39.5% Speed: 1.5ms Best for: Edge, mobile Performance: Speed 95 Accuracy 40 Size 10 YOLO('11n.pt')	Small (11s) Params: 9.4M mAP: 47.0% Speed: 2.5ms Best for: Embedded, real-time Performance: Speed 85 Accuracy 55 Size 25 YOLO('11s.pt')	Medium (11m) Params: 20.1M mAP: 51.5% Speed: 4.7ms Best for: Balanced (recommended) Performance: Speed 65 Accuracy 72 Size 50 YOLO('11m.pt')	Large (11l) Params: 25.3M mAP: 53.4% Speed: 6.2ms Best for: High accuracy Performance: Speed 50 Accuracy 85 Size 65 YOLO('11l.pt')	XLarge (11x) Params: 56.9M mAP: 54.7% Speed: 11.3ms Best for: Maximum accuracy Performance: Speed 25 Accuracy 95 Size 95 YOLO('11x.pt')
---	---	--	--	---

Recommendation: YOLO11m for our project — best balance of accuracy (51.5% mAP) and speed (4.7ms)

YOLO in Python — Image Detection

Three lines to detect objects. The Ultralytics API is beautifully simple.

Installation + Basic Detection

Install (one package does everything)

pip install ultralytics

from ultralytics import YOLO

Load pre-trained model (auto-downloads)

model = YOLO('yolo11m.pt')

Detect objects in an image

results = model('airport_frame.jpg')

Save annotated image

results[0].save('output.jpg')

That's it! Three lines for full detection.

Accessing Detection Results

for result in results:

for box in result.boxes:

Bounding box coordinates

x1,y1,x2,y2 = box.xyxy[0].tolist()

Confidence score (0-1)

conf = box.conf[0].item()

Class name

cls_id = int(box.cls[0])

name = model.names[cls_id]

print(f'{name}: {conf:.2f}')

Key Parameters

conf	0.25	Minimum confidence. Higher = fewer but more certain detections.
iou	0.7	NMS IoU threshold. Higher = stricter duplicate removal.
classes	None	Filter to specific class IDs: classes=[0, 2, 4, 7]
imgsz	640	Input size. Larger = slower but catches small objects.
device	auto	'cpu', 'cuda:0' (NVIDIA GPU), or 'mps' (Apple Silicon).
verbose	True	Print results to console. Set False for clean video loops.

YOLO on Video — Frame-by-Frame Detection

Same Day 3 video loop + YOLO detection per frame

Video Detection Loop

```
from ultralytics import YOLO
import cv2

model = YOLO('yolo11m.pt')
cap = cv2.VideoCapture('Airplane_Video.mp4')

while True:
    ret, frame = cap.read()
    if not ret: break

    # Run YOLO on this frame
    results = model(frame,
                    conf=0.5,
                    verbose=False)

    # Get annotated frame with boxes
    annotated = results[0].plot()

    cv2.imshow('YOLO', annotated)
    if cv2.waitKey(1) & 0xFF==ord('q'):
        break

cap.release()
```

Filtering to Airport-Relevant Classes

COCO has 80 classes — we only care about a few. Filter to avoid detecting cups, books, etc.:

```
# COCO class IDs we care about
AIRPORT_CLASSES = {
    0: 'person',
    2: 'car',
    4: 'airplane',
    5: 'bus',
    7: 'truck',
}

results = model(frame, conf=0.5,
                classes=list(AIRPORT_CLASSES.keys()))
```

Speed Tip: Skip Frames

Running YOLO on every frame is slow. For our project, detecting every 5th frame is enough:

```
frame_count = 0
while True:
    ret, frame = cap.read()
    frame_count += 1
    if frame_count % 5 == 0: # every 5th
        results = model(frame, conf=0.5)
```

Custom Training — Teaching YOLO New Objects

When pre-trained COCO classes aren't enough for your domain

Airport Objects — COCO Coverage

Airplane	✓ In COCO	Pre-trained detects it
Truck (generic)	✓ In COCO	Detected, but can't distinguish types
Person	✓ In COCO	Ground crew detection
Jet bridge	✗ NOT in COCO	Needs custom training
Fuel truck	✗ NOT in COCO	Needs custom training
Baggage cart	✗ NOT in COCO	Needs custom training
Pushback tug	✗ NOT in COCO	Needs custom training

Custom Training Pipeline

1 Collect

Extract video frames
+ find online images

2 Annotate

Draw bounding boxes
(Roboflow/LabelImg)

3 Organize

train/val/test splits
+ data.yaml config

4 Train

model.train(data=...)
transfer learning

5 Evaluate

mAP, precision, recall
on validation set

6 Deploy

Use best.pt for
inference

Transfer Learning — Why Start from Pre-Trained

Training from scratch requires millions of images
and weeks of GPU time.

Transfer learning starts from a model that already
understands visual features (edges, textures, shapes)
and only needs to learn the new classes.

Starting from yolo11m.pt, you only need
~100 images per class to get good results!

What Gets Transferred

Conv 1-3
Edges, textures

FROZEN
(already learned)

Conv 4-6
Patterns, shapes

FROZEN
(already learned)

Conv 7+
Object parts

FINE-TUNED
(adapted)

Head
New classes

TRAINED
(from scratch)

Custom Training — Dataset & Code

Organize your data, configure, train, evaluate

Dataset Structure

```
dataset/
├── data.yaml    # config
├── train/
│   ├── images/  # .jpg files
│   └── labels/   # .txt files
├── val/
│   ├── images/
│   └── labels/
└── test/
    ├── images/
    └── labels/
```

```
# data.yaml:
train: train/images
val: val/images
nc: 6 # number of classes
```

YOLO Annotation Format

Each image gets a .txt file with one line per object.
All values are normalized (0 to 1):

```
# class_id center_x center_y width height
0 0.50 0.40 0.30 0.20 ← airplane
1 0.20 0.70 0.10 0.15 ← fuel_truck
2 0.80 0.65 0.08 0.12 ← baggage_cart
```

Training & Evaluation Code

```
from ultralytics import YOLO

# Start from pre-trained (transfer learning)
model = YOLO('yolo11m.pt')

# Train on custom dataset
results = model.train(
    data='dataset/data.yaml',
    epochs=100,
    imgsz=640,
    batch=16,
    patience=20, # early stopping
    project='airport',
    name='turnaround_v1'
)
```

Annotation Tools

Roboflow

Free tier. Web-based. Auto-exports YOLO format. Smart annotation assists. roboflow.com

LabelImg

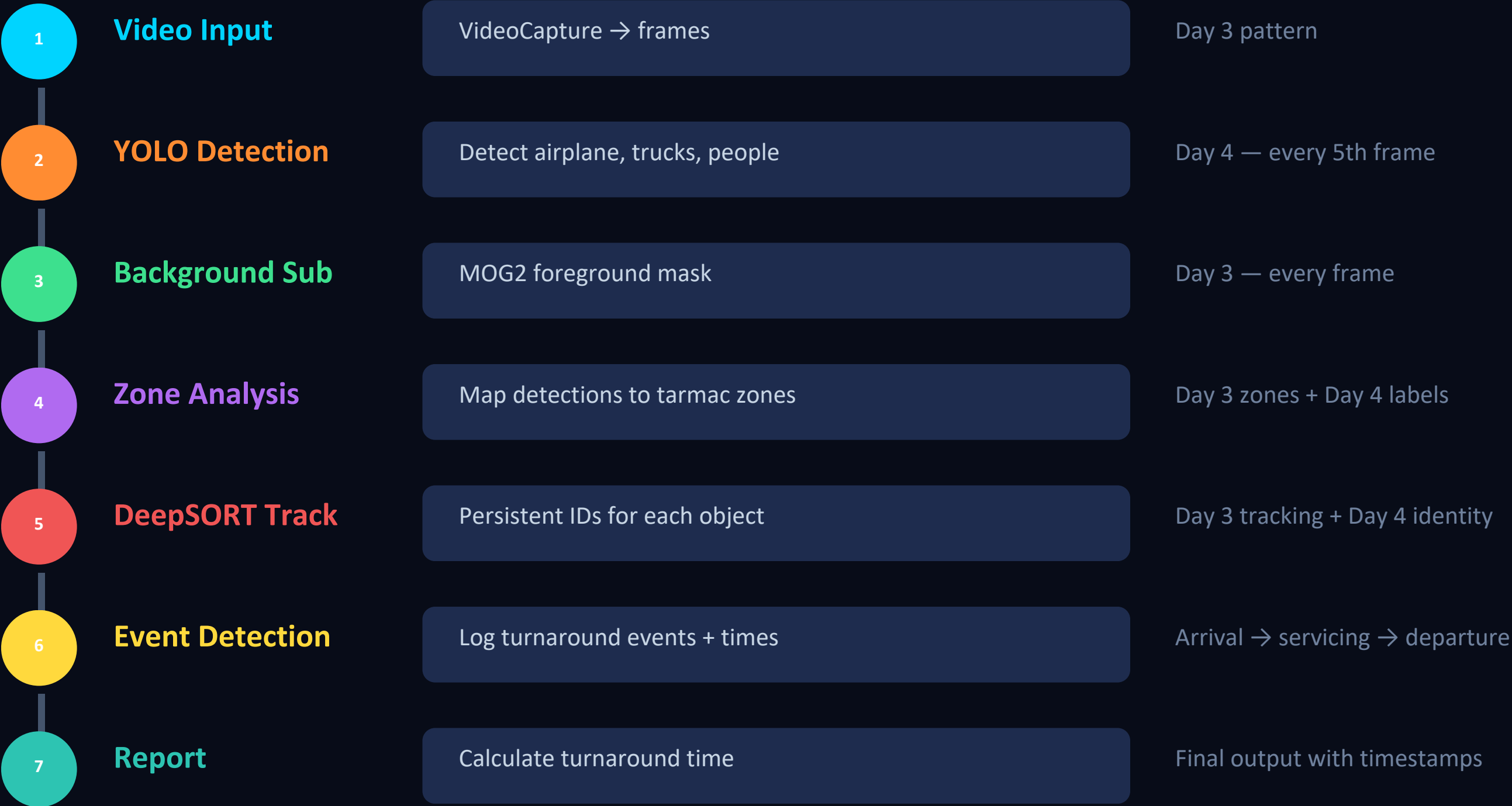
Open-source desktop tool.
`pip install labelImg`
Simple bounding box annotation.

CVAT

Intel's annotation tool. Self-hosted or cloud. Best for teams. cvat.ai

The Turnaround Pipeline — Full Architecture

Combining everything from Days 1-4 into one working system



Four Days of Building Blocks → One Pipeline

Day 1: Preprocess (grayscale, blur, resize) **Day 2:** Extract structure (edges, contours, features) **Day 3:** Motion + tracking (BG sub, optical flow, DeepSORT) **Day 4:** Intelligence (YOLO detection, classification, events)

Pipeline Code — TurnaroundAnalyzer Class

Core initialization and frame analysis loop

Class Setup

class TurnaroundAnalyzer:

```
def __init__(self, video_path):
    self.cap = cv2.VideoCapture(video_path)
    self.fps = self.cap.get(CAP_PROP_FPS)
    self.model = YOLO('yolo11m.pt')
    self.bg_sub = cv2.create..MOG2(
        history=500, varThreshold=50)
    self.kernel = getStructuringElement(
        MORPH_ELLIPSE, (5,5))
    self.events = []
    self.motion_history = []
```

def analyze_frame(self, frame):

```
self.frame_count += 1
time = self.frame_count / self.fps
```

YOLO every 5th frame

```
detections = []
```

```
if self.frame_count % 5 == 0:
```

```
    results = self.model(frame,
        conf=0.4, classes=[0,2,4,5,7])
```

parse results...

Background Sub + Events

Background subtraction (every frame)

```
fg = self.bg_sub.apply(frame)
_, fg = cv2.threshold(
    fg, 250, 255, THRESH_BINARY)
fg = morphologyEx(fg, OPEN, self.kernel)
fg = morphologyEx(fg, CLOSE, self.kernel)
```

Motion level

```
motion = np.sum(fg>0)/fg.size*100
```

Event detection

```
airplane = [d for d in dets
    if d['class']=='airplane']
if airplane and not self.arrived:
    self.events.append({
        'time': time,
        'event': 'Aircraft at gate'})
```

Check for departure

```
if motion > 15 and self.servicing_done:
    self.events.append({
        'time': time,
        'event': 'Pushback detected'})
```

```
return detections, fg, motion
```

Performance Optimization — Speed & Accuracy

Practical techniques to make your pipeline faster and better

Speed Optimization

Skip frames

```
if frame_count % 5 == 0
```

75

5-10×

```
if frame_count % 5 == 0
```

Smaller input

```
model(frame, imgsz=320)
```

40

2-4×

```
model(frame, imgsz=320)
```

Smaller model

```
Use yolo11n.pt (Nano)
```

30

2-3×

```
Use yolo11n.pt (Nano)
```

GPU acceleration

```
model(frame, device='cuda:0')
```

95

10-50×

```
model(frame,  
device='cuda:0')
```

Half precision

```
model(frame, half=True) on GPU
```

20

1.5-2×

```
model(frame, half=True) on  
GPU
```

Batch processing

```
Process multiple frames at once
```

30

2-3×

```
Process multiple frames at  
once
```

Accuracy Optimization

Higher confidence

```
model(frame, conf=0.6)
```

Fewer false positives

Class filtering

```
classes=[0, 2, 4, 7]
```

Remove irrelevant

Larger input size

```
model(frame, imgsz=1280)
```

Better small objects

Larger model

```
Use yolo11l.pt or yolo11x.pt
```

Higher mAP

Custom training

```
Train on airport images
```

Domain accuracy

Test-time augment

```
model(frame, augment=True)
```

1-2% mAP boost

Real-World Airport CV Systems

Computer vision is already revolutionizing ground operations

Assaia ApronAI

Deployed at 20+ airports, 168+ airlines globally.
Cameras at each gate auto-detect ground events.

Results:

- Alaska Airlines: 17% increase in on-time performance
- Toronto Pearson: 3.4 min less ground delay per flight
- Real-time alerts for operational deviations
- Predicts departure readiness in advance

Schiphol Deep Turnaround

Amsterdam Airport's camera-based system.
Detects when ground handling starts and stops.

Capabilities:

- Predicts aircraft departure readiness
- Enables proactive delay prevention
- Time error < 60 seconds (A-CDM standard)
- Integrates with airline ops systems

Our Project vs Production Systems

Aspect	Our Course Project	Production System
Cameras	Single video file	4-8 cameras per gate, 24/7
Model	Pre-trained YOLO	Custom-trained on millions of images
Accuracy	Good enough to demonstrate	Must meet A-CDM (<60s error)
Speed	Offline processing OK	Real-time (30+ FPS minimum)
Infrastructure	Your laptop	Edge GPU + cloud pipeline
Output	Console report + print	API integration with airline ops

Your project demonstrates the same fundamental approach used by these multi-million dollar systems!



Day 4 — Key Takeaways

1

Classical CV detects motion; DL identifies objects

Combined: 'something moved' + 'it's a fuel truck.' Both needed for the project.

2

YOLO processes entire image in one pass

Grid prediction → all detections at once. That's why it's fast enough for real-time video.

3

Pre-trained YOLO knows 80 COCO classes

airplane, truck, car, person, bus are built-in. Jet bridge, fuel truck need custom training.

4

Transfer learning: ~100 images per class is enough

Start from yolo11m.pt → only teach new classes. Saves weeks of training time.

5

Ultralytics API: 3 lines for full detection

```
model = YOLO('yolo11m.pt') → results = model(image) → results[0].plot()
```

6

YOLO + DeepSORT = persistent tracked identities

YOLO detects per-frame, DeepSORT maintains identity across frames. No ID swaps.

7

Your pipeline combines all 4 days of techniques

Preprocess (D1) → Features (D2) → Motion+Track (D3) → Detection+Events (D4) = turnaround time.